

Python
for DA

Revision. Визуализация



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение



Серии в Pandas



DataFrame в Pandas



Очистка данных в Pandas



Слияние и объединение в Pandas



Группировка данных и сводные таблицы в Pandas

План занятия

- Основы построения графиков с помощью Matplotlib
- Настройка графиков
- Подграфики
- Основы построения графиков с Seaborn
- Парные графики и тепловые карты в Seaborn



ОСНОВНОЙ БЛОК





Основы построения графиков с помощью Matplotlib



Matplotlib

Это универсальная библиотека для создания широкого спектра статических, анимированных и интерактивных графиков на Python.



Линейные графики

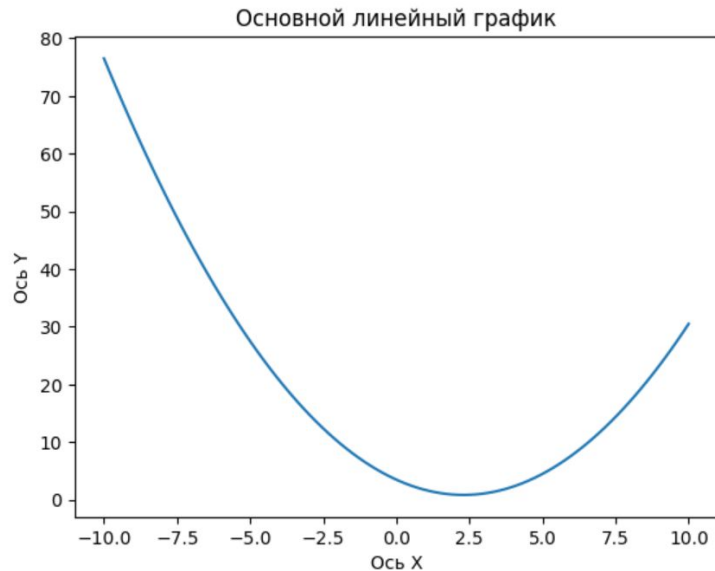
Используются для визуализации тенденций изменения данных за интервалы времени.

```
import matplotlib.pyplot as plt
import numpy as np

# Создаем квадратичную кривую
x = np.linspace(-10, 10, 100)
y = 3.5 - 2.3 * x + 0.5 * x ** 2

# Строим график данных
plt.plot(x, y)
plt.xlabel('Ось X')
plt.ylabel('Ось Y')
plt.title('Основной линейный график')
plt.show()
```

Создание базового линейного графика в Matplotlib



Результат



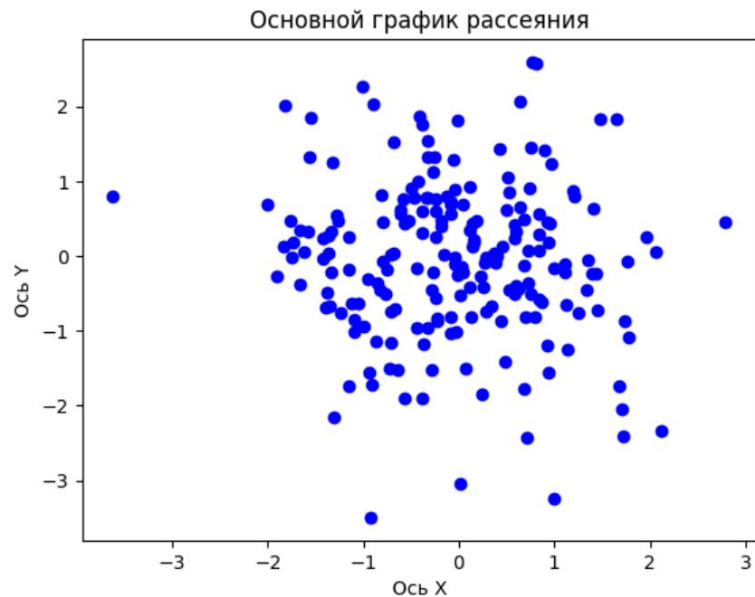
Графики рассеяния

Используются для отображения значений, как правило, двух переменных для набора данных.

```
# Генерируем случайные данные
x = np.random.randn(200)
y = np.random.randn(200)

# Создаём диаграмму рассеяния
plt.scatter(x, y, color='blue', marker='o')
plt.xlabel('Ось X')
plt.ylabel('Ось Y')
plt.title('Основной график рассеяния')
plt.show()
```

Создание графика рассеяния



Результат



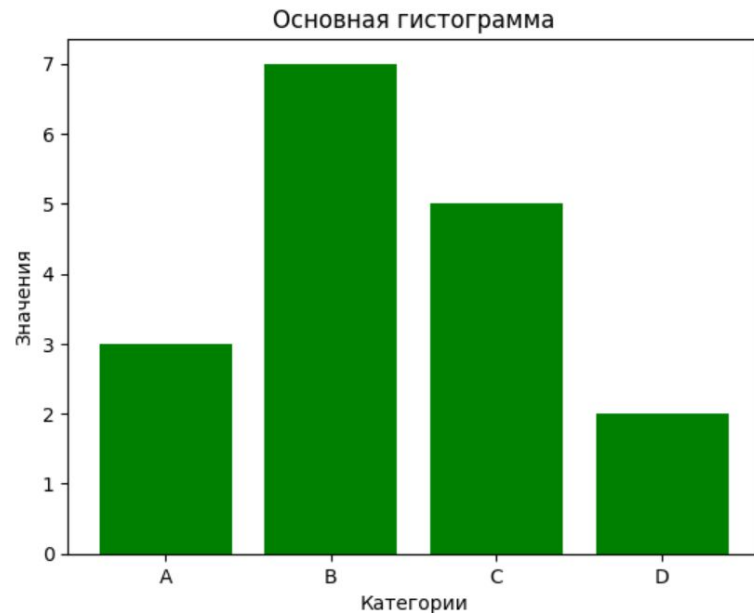
Столбчатые диаграммы

Используются для сравнения величин, относящихся к разным категориям.

```
# Определяем категории и значения
categories = ['A', 'B', 'C', 'D']
values = [3, 7, 5, 2]

# Создаём гистограмму
plt.bar(categories, values, color='green')
plt.xlabel('Категории')
plt.ylabel('Значения')
plt.title('Основная гистограмма')
plt.show()
```

Создание столбчатой диаграммы



Результат



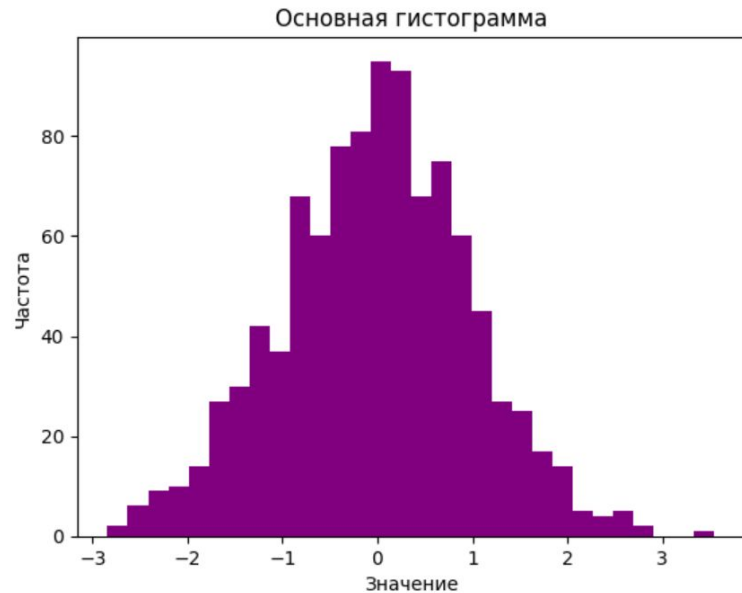
Гистограммы

Используются для представления распределения набора данных.

```
# Генерируем случайные данные
data = np.random.randn(1000)

# Создаём гистограмму
plt.hist(data, bins=30, color='purple')
plt.xlabel('Значение')
plt.ylabel('Частота')
plt.title('Основная гистограмма')
plt.show()
```

Создание гистограммы



Результат



ВОПРОСЫ





Настройка графиков

Заголовки

```
import matplotlib.pyplot as plt
import numpy as np

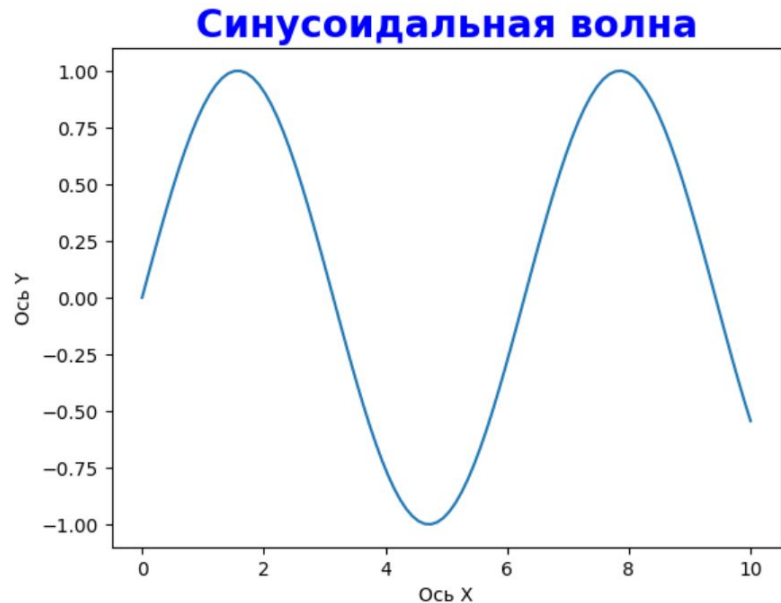
# Данные выборки
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Построение графика данных
plt.plot(x, y)

# Настройка заголовка
plt.title('Синусоидальная волна', fontsize=20,
fontweight='bold', color='blue')
plt.xlabel('Ось X')
plt.ylabel('Ось Y')

# Отображение графика
plt.show()
```

Добавление заголовка



Результат

Метки

```
import matplotlib.pyplot as plt
import numpy as np

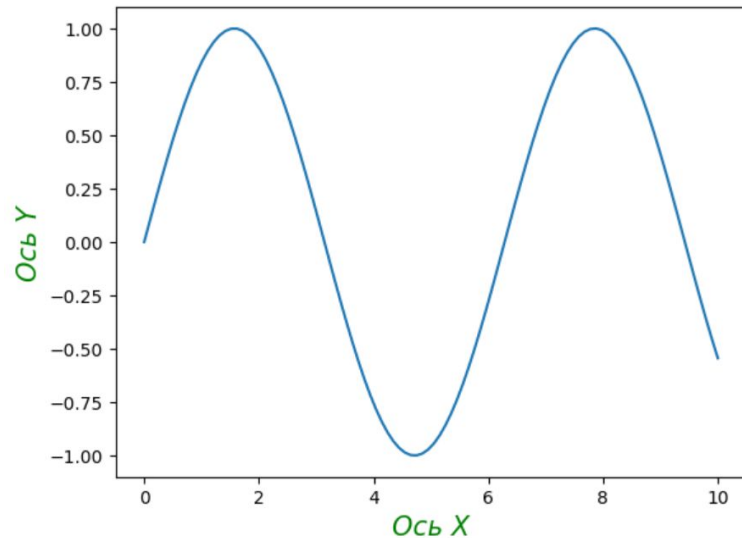
# Данные выборки
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Построение графика данных
plt.plot(x, y)

# Настройка меток
plt.xlabel('Ось X', fontsize=15,
           fontstyle='italic', color='green')
plt.ylabel('Ось Y', fontsize=15,
           fontstyle='italic', color='green')

plt.show()
```

Добавление меток



Результат

Легенды

```
# Примерные данные
y1 = np.sin(x)
y2 = np.cos(x)

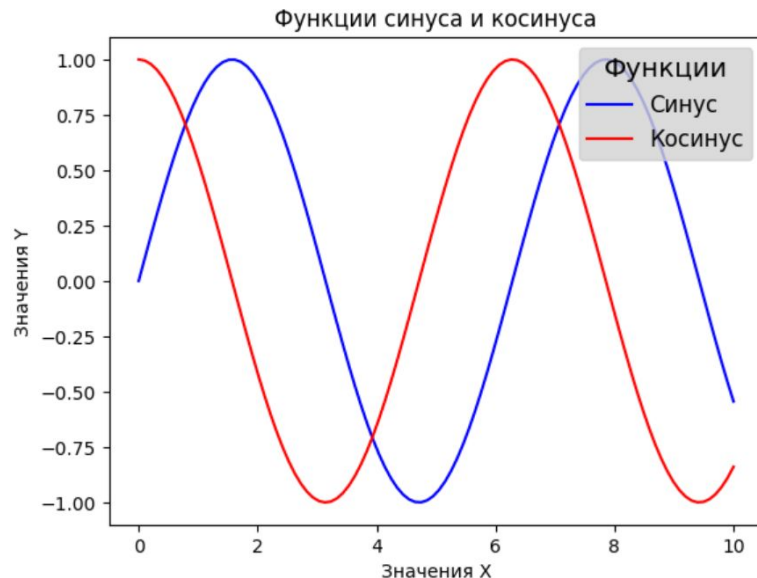
# Построение графика с метками и цветами
plt.plot(x, y1, label='Синус', color='blue')
plt.plot(x, y2, label='Косинус', color='red')

# Настройка легенды
plt.legend(loc='upper right', fontsize='large',
title='Функции', title_fontsize='x-large',
facecolor='lightgrey', labelcolor='black')

# Добавление деталей
plt.title('Функции синуса и косинуса')
plt.xlabel('Значения X')
plt.ylabel('Значения Y')

# Отображение графика
plt.show()
```

Добавление легенды



Результат

Цвета

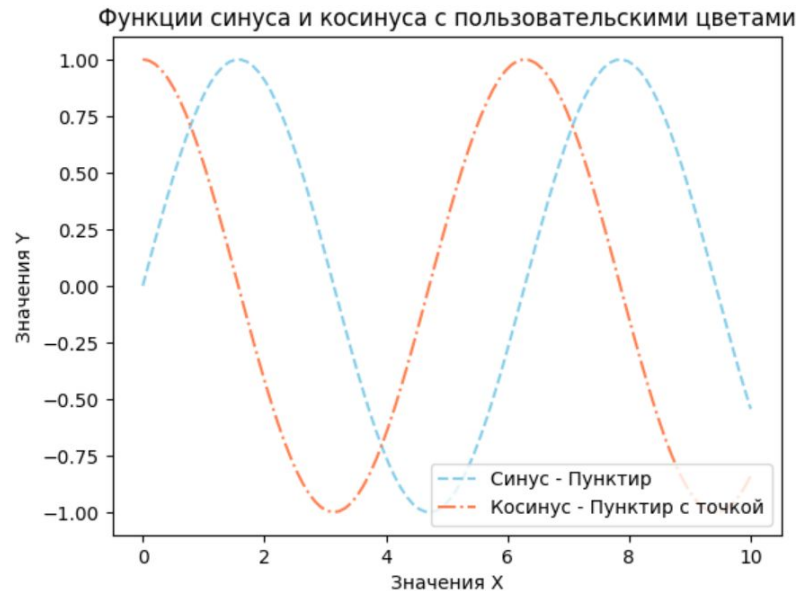
```
# Построение графиков с использованием
# пользовательских цветов
plt.plot(x, y1, label='Синус', color='skyblue',
linestyle='--')
plt.plot(x, y2, label='Косинус', color='coral',
linestyle='-.')

# Настройка легенды
plt.legend(labels=['Синус - Пунктир', 'Косинус -
Пунктир с точкой'], loc='lower right',
fontsize='medium')

# Добавление деталей
plt.title('Функции синуса и косинуса с
пользовательскими цветами')
plt.xlabel('Значения X')
plt.ylabel('Значения Y')

# Отображение графика
plt.show()
```

Добавление цветов



Результат



ВОПРОСЫ





Подграфики



Подграфики

Это функция в Matplotlib, позволяющая создавать фигуры, содержащие несколько графиков, которые могут быть расположены в различных макетах.

Функция plt.subplots()

```
import matplotlib.pyplot as plt
import numpy as np

# Создаем примерные данные
x = np.linspace(0, 2 * np.pi, 400)
y = np.sin(x ** 2)

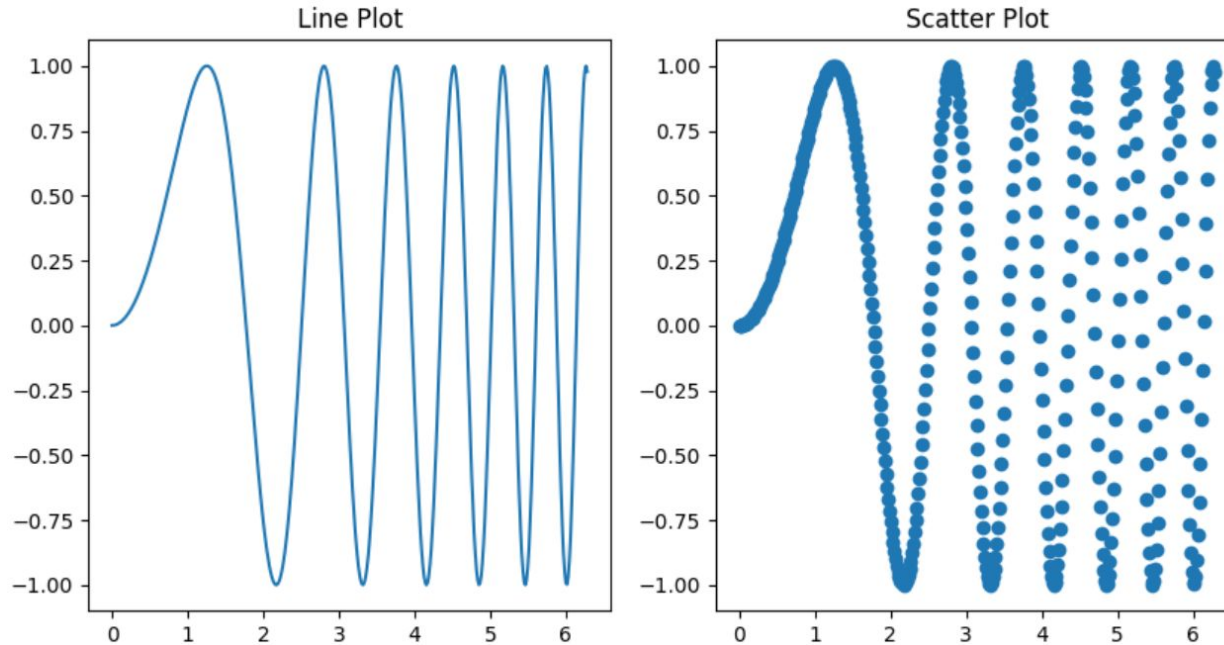
# Создайте рисунок с двумя подплощадками
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))

# Нанесите данные на первый подграфик
ax1.plot(x, y)
ax1.set_title('Line Plot')

# Постройте данные на втором подграфике
ax2.scatter(x, y)
ax2.set_title('Scatter Plot')

# Отобразите рисунок
plt.show()
```

Функция plt.subplots()





GridSpec

Позволяет гибко позиционировать подграфики в сетке. Подграфики могут охватывать несколько строк или столбцов.

GridSpec

```
from matplotlib.gridspec import GridSpec

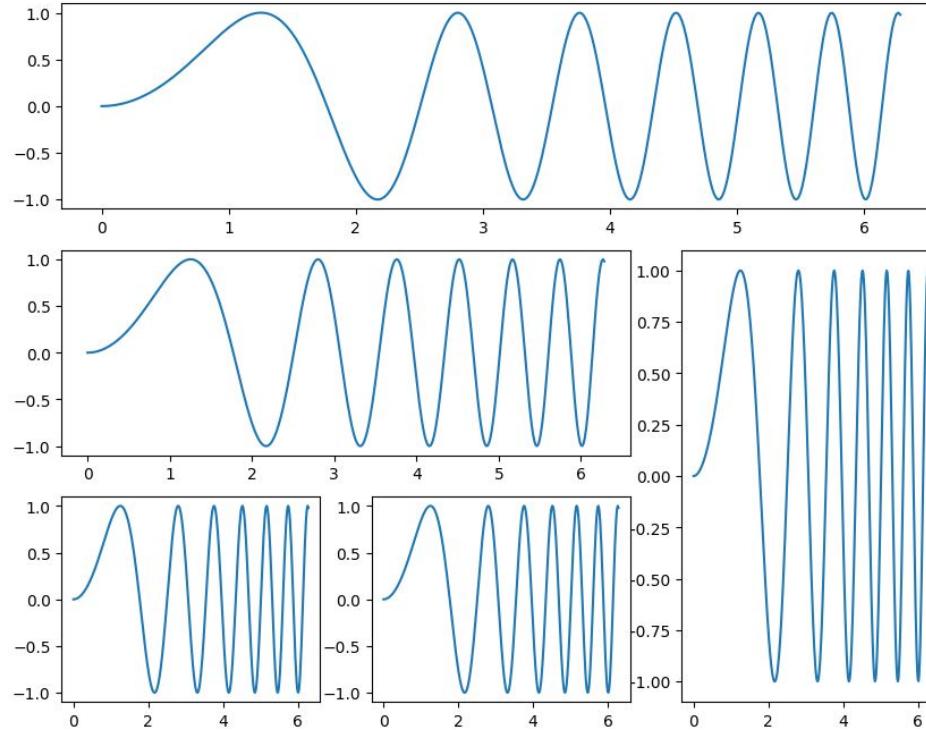
fig = plt.figure(figsize=(10, 8))
gs = GridSpec(3, 3, figure=fig)

ax1 = fig.add_subplot(gs[0, :])
ax2 = fig.add_subplot(gs[1, :-1])
ax3 = fig.add_subplot(gs[1:, -1])
ax4 = fig.add_subplot(gs[-1, 0])
ax5 = fig.add_subplot(gs[-1, -2])

# Построение данных на подграфиках
ax1.plot(x, y)
ax2.plot(x, y)
ax3.plot(x, y)
ax4.plot(x, y)
ax5.plot(x, y)

plt.show()
```

GridSpec





`subplot_mosaic`

Позволяет задавать структуру фигуры с помощью форматированных строк.

subplot_mosaic

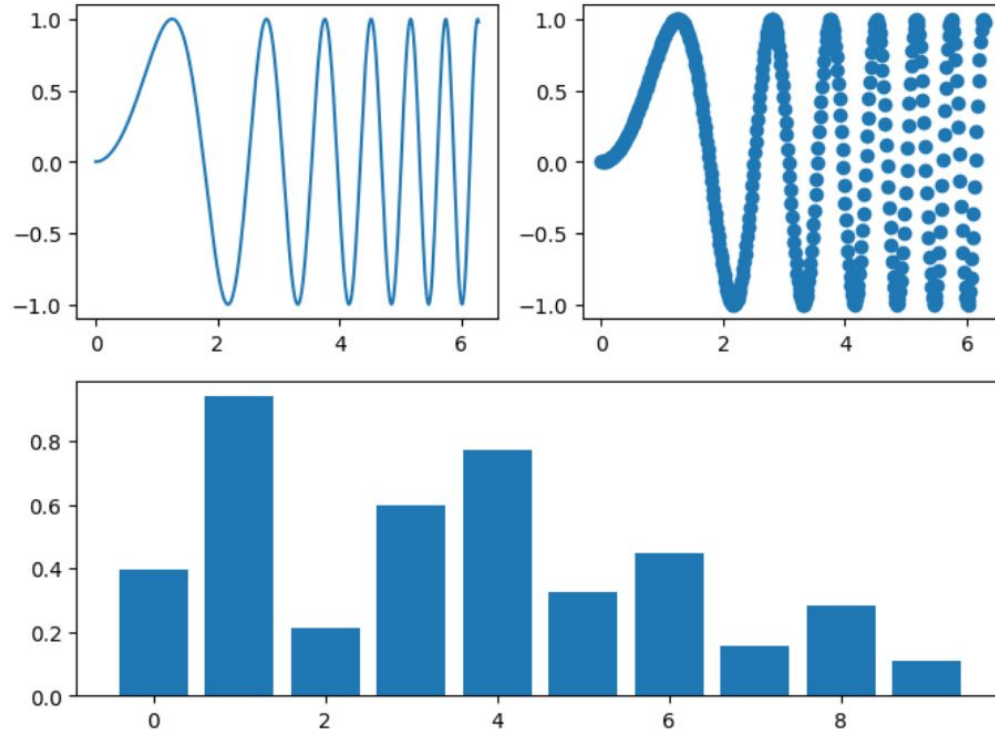
```
layout = """
AB
CC
"""

fig, axes = plt.subplot_mosaic(layout, figsize=(8, 6))

axes['A'].plot(x, y)
axes['B'].scatter(x, y)
axes['C'].bar(np.arange(10), np.random.rand(10))

plt.show()
```


subplot_mosaic





ВОПРОСЫ





Основы построения графиков с Seaborn



Seaborn

Это библиотека Python, построенная поверх Matplotlib и предназначенная для создания привлекательных и информативных статистических графиков.



Графики распределения

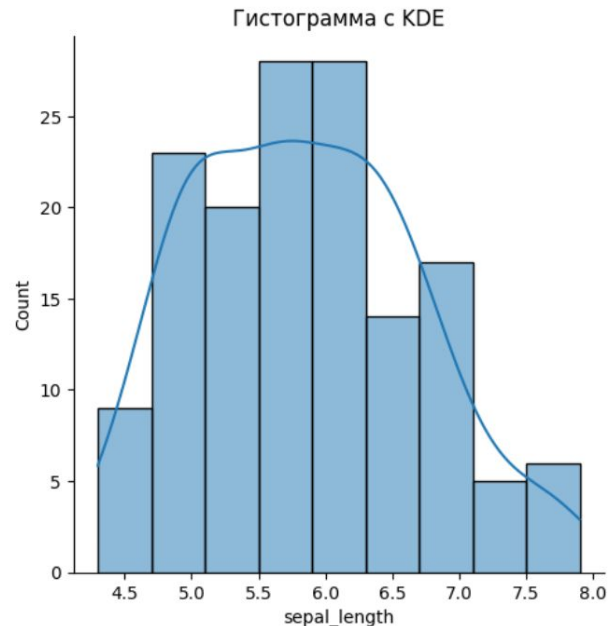
Используются для визуализации распределения набора данных.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Загружаем данные выборки
data = sns.load_dataset("iris")

# Гистограмма с помощью KDE
sns.displot(data['sepal_length'], kde=True)
plt.title('Гистограмма с KDE')
plt.show()
```

Создание графика распределения



Результат

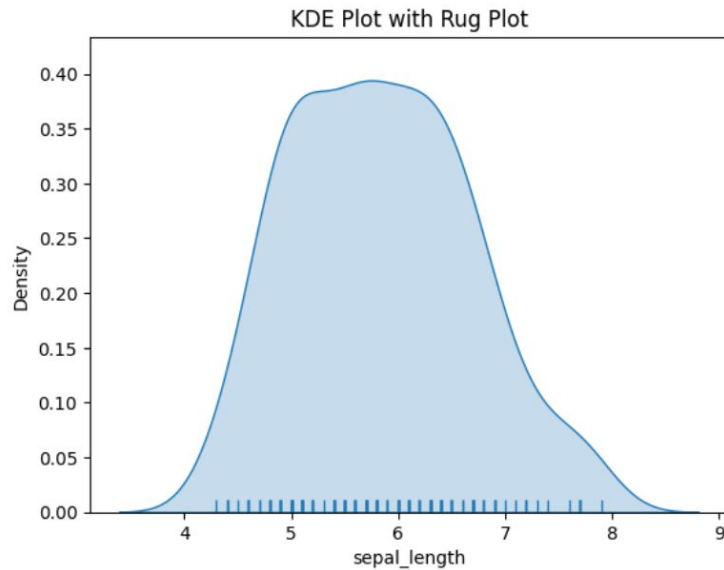


Rug графики

Добавляют небольшие вертикальные линии в каждой точке данных вдоль оси x , обеспечивая визуальное представление распределения данных.

```
sns.kdeplot(data['sepal_length'], fill=True)
sns.rugplot(data['sepal_length'])
plt.title('KDE Plot with Rug Plot')
plt.show()
```

Создание rug графика



Результат

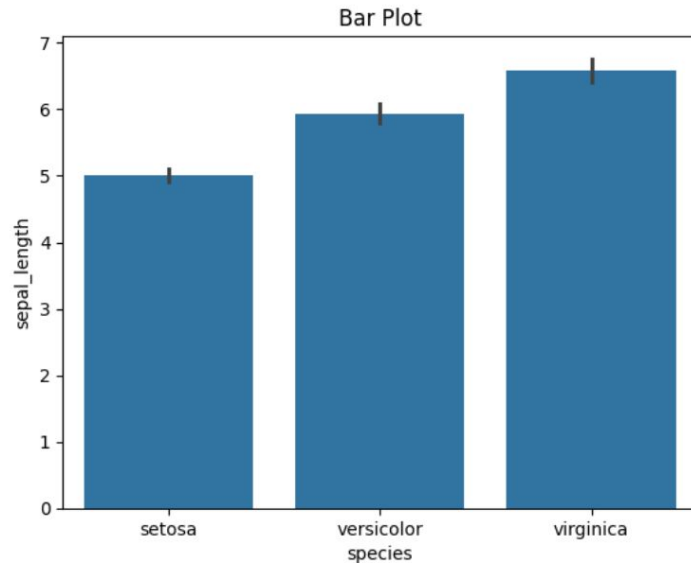


Категориальные графики

Используются для визуализации данных, в которых одна или несколько переменных являются категориальными.

```
# Гистограмма
sns.barplot(x='species', y='sepal_length',
data=data)
plt.title('Bar Plot')
plt.show()
```

Создание категориального графика



Результат

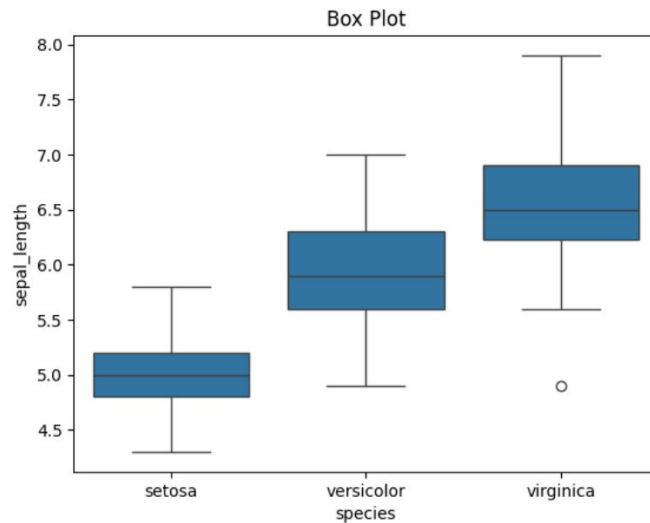


Графики ящик с усами

Отображают распределение непрерывной переменной для каждой категории категориальной переменной.

```
# Коробчатый график
sns.boxplot(x='species', y='sepal_length',
data=data)
plt.title('Box Plot')
plt.show()
```

Создание графика ящик с усами



Результат

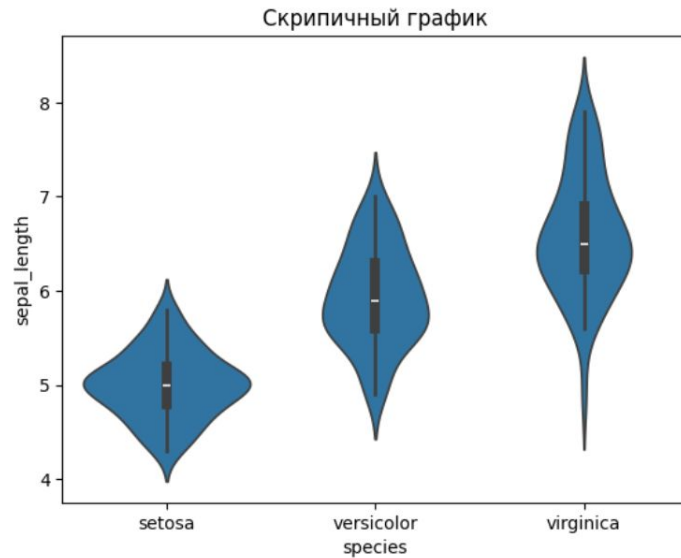


Скрипичные графики

Полезны для сравнения распределения непрерывной переменной по разным категориям.

```
# Скрипичный график
sns.violinplot(x='species', y='sepal_length',
data=data)
plt.title('Скрипичный график')
plt.show()
```

Создание скрипичного графика



Результат

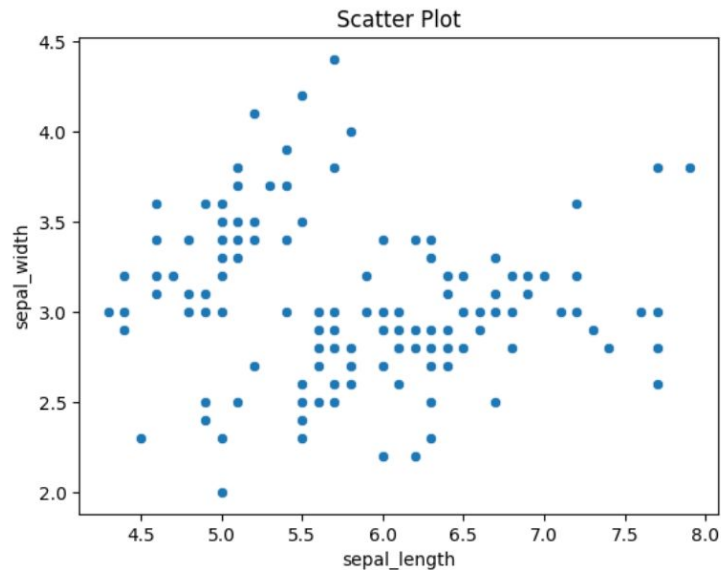


Реляционные графики

Используются для визуализации взаимосвязи между двумя или более переменными.

```
# Диаграмма рассеяния
sns.scatterplot(x='sepal_length',
y='sepal_width', data=data)
plt.title('Scatter Plot')
plt.show()
```

Создание реляционного графика



Результат

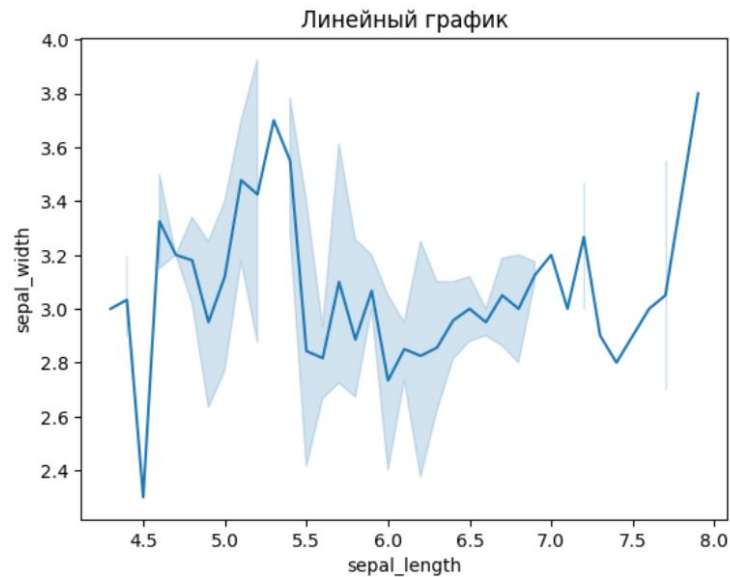


Линейные графики

Полезны для визуализации тенденций с течением времени или другой непрерывной переменной.

```
# Линейный график
sns.lineplot(x='sepal_length',
y='sepal_width', data=data)
plt.title('Линейный график')
plt.show()
```

Создание линейного графика



Результат



Реляционные графики

Полезны для изучения и понимания взаимосвязей между несколькими переменными в наборе данных.



ВОПРОСЫ





Парные графики и тепловые карты в Seaborn



Парные графики

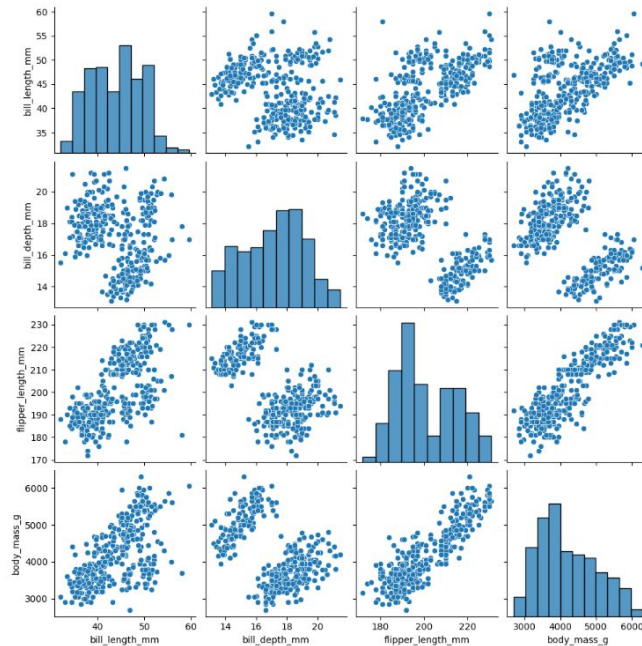
Используются для визуализации парных отношений в наборе данных. Полезны для изучения взаимосвязей между несколькими переменными одновременно.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Загрузка данных выборки
penguins = sns.load_dataset("penguins")

# Создайте парный график
sns.pairplot(penguins)
plt.show()
```

Создание парных графиков



Результат

Парные графики

```
# Парный график с оттенком
sns.pairplot(penguins, hue="species")
plt.show()
```

```
# Парный график с различными видами графиков
sns.pairplot(penguins, kind="kde")
plt.show()
```

```
# Парный график с пользовательскими маркерами и размерами
sns.pairplot(penguins, hue="species", markers=["o", "s", "D"], height=2.5)
plt.show()
```




Тепловые карты

Полезны для визуализации корреляционной матрицы между несколькими переменными, позволяя с первого взгляда определить высокоррелированные или обратнокоррелированные переменные.

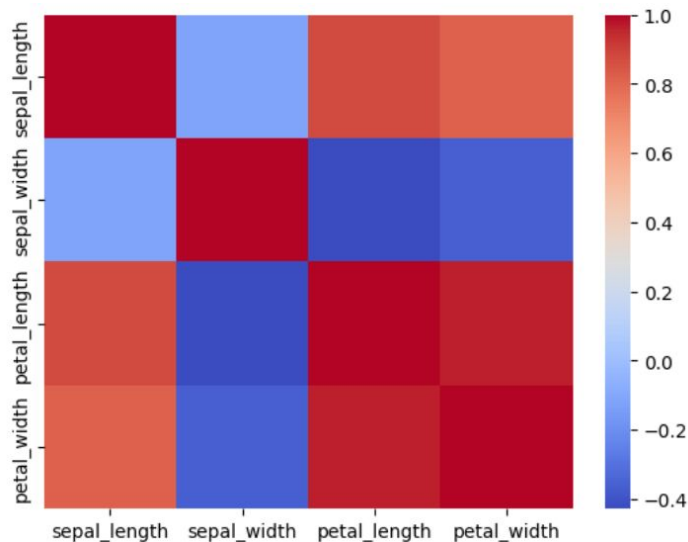
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Загрузка данных выборки
data = sns.load_dataset("iris")
data = data[["sepal_length", "sepal_width",
"petal_length", "petal_width"]]

# Рассчитайте корреляционную матрицу
correlation_matrix = data.corr()

# Создайте тепловую карту
sns.heatmap(correlation_matrix,
cmap='coolwarm')
plt.show()
```

Создание тепловых графиков



Результат



ВОПРОСЫ



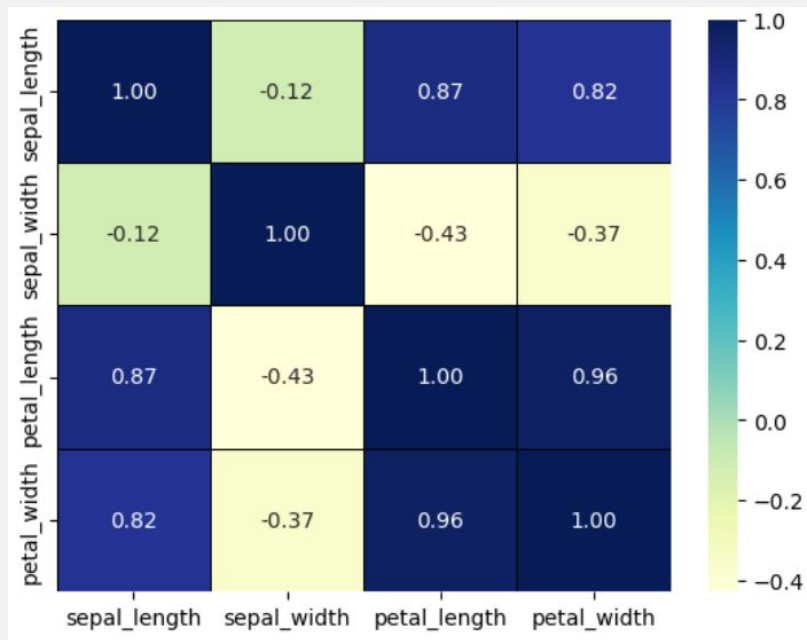


ПРАКТИЧЕСКАЯ РАБОТА



Задание

1. Повторите график по датасету Iris:





ВОПРОСЫ



Заключение

